

BÀI 4. VIEW COMPONENTS VÀ KỸ THUẬT PHÂN TRANG

Tuần trước, chúng ta đã tạo xong giao diện cơ bản. Tuy nhiên, ứng dụng của chúng ta vẫn còn khá thô sơ: sidebar là tĩnh và nếu có 1000 sản phẩm, trang web sẽ cố gắng tải tất cả, dẫn đến trải nghiệm người dùng cực kỳ tệ. Tuần này, chúng ta sẽ dạy cho ứng dụng cách trở nên **thông minh, hiệu quả, và có tổ chức** hơn.

MỤC TIÊU BÀI HỌC

Sau khi hoàn thành tuần học này, bạn sẽ có khả năng:

1. **Phân biệt rành mạch** khi nào nên dùng Partial View và khi nào phải dùng View Component.
2. **Tự tay xây dựng** một View Component hoàn chỉnh để tạo ra các khối UI độc lập và tái sử dụng.
3. **Sử dụng thành thạo** các phương thức Skip() và Take() của LINQ để triển khai logic phân trang hiệu quả.
4. **Tạo một Tag Helper tùy chỉnh** để đóng gói logic phức tạp, giúp mã nguồn trong View trở nên sạch sẽ và dễ đọc hơn.

NỘI DUNG LÝ THUYẾT

4.1. View Components - Những "Ứng dụng mini" trong trang web

4.1.1. View Component là gì?

Hãy tưởng tượng một View Component như một **"mini-controller"** độc lập. Nó là một khối giao diện người dùng (UI) có vòng đời riêng, có thể chứa logic nghiệp vụ phức tạp của riêng nó (như gọi service, truy vấn CSDL), và hoàn toàn có thể tái sử dụng ở bất kỳ đâu trong ứng dụng.

Ví dụ: Một giỏ hàng tóm tắt ở góc trên cùng, một menu danh mục, một form đăng nhập... Đây là những thành phần mà logic của chúng (lấy dữ liệu giỏ hàng, lấy danh sách danh mục) không liên quan trực tiếp đến nội dung chính của trang đang hiển thị.

4.1.2. So sánh với Partial View: "Bức tranh" vs. "Cửa sổ thông minh"

Đây là một điểm rất quan trọng mà nhiều người mới học hay nhầm lẫn.

Tiêu chí	Partial View (_Partial.cshtml)	View Component
Bản chất	Một "bức tranh" tĩnh.	Một "cửa sổ thông minh".
Logic	Không có logic C# riêng. Nó chỉ nhận dữ liệu được "mớm" sẵn từ View cha.	Có logic C# riêng. Nó có thể tự đi lấy dữ liệu của mình (gọi service, truy vấn CSDL).
Sự độc lập	Phụ thuộc hoàn toàn vào Controller của View cha.	Hoàn toàn độc lập. Nó không quan tâm Controller nào đang gọi nó.
Khi nào dùng	Khi bạn cần tái sử dụng một đoạn markup (HTML) đơn giản ở nhiều nơi. VD: một card sản phẩm.	Khi bạn cần tái sử dụng một khối UI có logic nghiệp vụ phức tạp và cần tự lấy dữ liệu. VD: menu danh mục, giỏ hàng tóm tắt.

4.1.3. Cấu trúc của một View Component

Một View Component luôn bao gồm 2 phần:

1. Lớp Logic (File .cs):

- Tên lớp phải kết thúc bằng ViewComponent (quy ước).
- Phải kế thừa từ lớp ViewComponent của ASP.NET Core.
- Phải có một phương thức tên là Invoke hoặc InvokeAsync. Đây là "Action" của View Component. Nó sẽ được gọi khi bạn render component.
- Phương thức này chịu trách nhiệm xử lý logic và trả về một IActionResultResult (thường là View(data)).

2. Giao diện (File .cshtml):

- Một file Razor bình thường, thường được đặt tên là Default.cshtml.
- Được đặt trong một cấu trúc thư mục quy ước: Views/Shared/Components/{TênComponent}/Default.cshtml.

– **Áp dụng vào SportsStore:** Tại sao menu danh mục là ứng viên hoàn hảo?

- **Logic độc lập:** Việc lấy danh sách các danh mục sản phẩm từ CSDL là một

ng nghiệp vụ hoàn toàn độc lập với việc hiển thị danh sách sản phẩm (có thể được phân trang, sắp xếp...).

- **Tách biệt mối quan tâm (SoC):** HomeController chỉ nên quan tâm đến việc xử lý sản phẩm. Việc lấy danh mục nên thuộc về một thành phần riêng. Nếu không dùng View Component, bạn sẽ phải viết code lấy danh mục trong *mọi Action* cần hiển thị sidebar, gây trùng lặp code và vi phạm nguyên tắc SoC.

4.2. Phân trang với LINQ - "Cắt lát" dữ liệu khổng lồ

4.2.1. Tại sao cần phân trang?

Hãy tưởng tượng bạn có một CSDL với 10,000 sản phẩm. Nếu HomeController của bạn lấy tất cả 10,000 sản phẩm, sau đó truyền cho View để hiển thị:

1. **Server:** Sẽ tốn rất nhiều tài nguyên để truy vấn và kéo 10,000 bản ghi về bộ nhớ.
2. **Đường truyền:** Dữ liệu truyền từ server về client sẽ cực lớn.
3. **Client (Trình duyệt):** Sẽ "treo" khi cố gắng render một trang HTML chứa 10,000 sản phẩm.

⇒ **Kết quả:** Ứng dụng siêu chậm, trải nghiệm người dùng bằng không.

Phân trang là kỹ thuật chia tập dữ liệu khổng lồ đó thành các "lát" (trang) nhỏ hơn và chỉ tải về "lát" mà người dùng đang yêu cầu.

4.2.2. Giới thiệu LINQ (Language Integrated Query)

LINQ là một công nghệ mang tính cách mạng trong hệ sinh thái .NET. Trước khi có LINQ, lập trình viên phải học và sử dụng nhiều ngôn ngữ truy vấn khác nhau tùy thuộc vào nguồn dữ liệu (ví dụ: dùng T-SQL cho database SQL Server, dùng XPath/XQuery cho file XML, và dùng các vòng lặp for/foreach thủ công cho các danh sách List/Array trong bộ nhớ). LINQ cho phép bạn viết các câu lệnh truy vấn dữ liệu (từ CSDL, XML, Collections...) bằng cú pháp C# quen thuộc, thay vì phải viết chuỗi SQL. LINQ ra đời để giải quyết sự phân mảnh này bằng cách cung cấp một **ngôn ngữ truy vấn thống nhất** ngay bên trong C#.

1. Tại sao LINQ lại là một "Đột phá"?

- **Đồng nhất cú pháp:** Cho dù bạn đang truy vấn một List<Product> trong RAM, một bảng Products trong SQL Server, hay một file products.xml, bạn chỉ cần

dùng một kiểu cú pháp LINQ duy nhất.

- **An toàn kiểu dữ liệu (Type Safety):** Khác với việc viết các câu SQL dưới dạng chuỗi văn bản (string) dễ dẫn đến lỗi sai chính tả và chỉ phát hiện được khi ứng dụng chạy (Runtime), LINQ được kiểm tra lỗi ngay lúc biên dịch (Compile-time). Nếu bạn gõ sai tên cột, Visual Studio sẽ báo lỗi đỏ ngay lập tức.
- **Hỗ trợ IntelliSense:** Visual Studio sẽ tự động gợi ý tên bảng, tên thuộc tính, các phương thức khi bạn gõ code LINQ, giúp tăng tốc độ gõ và giảm thiểu sai sót.

2. Hai phong cách viết mã LINQ

Khi sử dụng LINQ, bạn có hai cách viết (cú pháp) để tạo ra cùng một kết quả. Trình biên dịch sẽ tự động biên dịch cả hai về cùng một mã máy.

Cách 1: Query Syntax (Cú pháp truy vấn)

Cú pháp này được thiết kế trông rất giống với SQL truyền thống (nhưng viết ngược lại: from đứng trước, select đứng cuối). Rất dễ làm quen cho những ai đã biết SQL.

```
// Giả sử có một danh sách sản phẩm
List<Product> products = GetProductList();

// Lấy ra các sản phẩm có giá > 100, sắp xếp theo tên
var expensiveProducts = from p in products
                        where p.Price > 100
                        orderby p.Name
                        select p;
```

Cách 2: Method Syntax (Cú pháp phương thức - Khuyến dùng)

Sử dụng các phương thức mở rộng (Extension Methods) kết hợp với **Biểu thức Lambda (Lambda Expressions - =>)**. Đây là cú pháp phổ biến nhất, ngắn gọn nhất và thường được dùng xuyên suốt trong các dự án ASP.NET Core / Entity Framework Core hiện đại.

```
// Cùng logic với Query Syntax ở trên
var expensiveProducts = products
    .Where(p => p.Price > 100)
    .OrderBy(p => p.Name);
```

(Đọc là: Lọc danh sách products, lấy ra những sản phẩm (gọi là p) sao cho p.Price lớn hơn 100, sau đó sắp xếp theo p.Name).

3. Đặc tính cực kỳ quan trọng: Thực thi trì hoãn (Deferred Execution)

Khi bạn viết một câu lệnh LINQ như `var query = products.Where(...)`, câu lệnh này **chưa hề được chạy**. Quá trình lọc dữ liệu không xảy ra ngay lập tức. Câu lệnh LINQ lúc này chỉ đóng vai trò là một "bản vẽ thiết kế".

Câu lệnh LINQ chỉ thực sự được mang ra thực thi (truy vấn vào DB hoặc chạy qua danh sách) khi bạn bắt đầu **duyệt qua kết quả** của nó. Thông thường, điều này xảy ra khi bạn:

- Dùng vòng lặp `foreach` để duyệt qua biến `query`.
- Gọi các phương thức ép kiểu về danh sách cụ thể như `.ToList()`, `.ToArray()`.
- Gọi các phương thức lấy phần tử đơn như `.FirstOrDefault()`, `.Count()`.

Tính năng này giúp tối ưu hóa hiệu suất rất tốt, cho phép bạn nối (chain) nhiều điều kiện LINQ lại với nhau trước khi thực sự gửi một câu truy vấn duy nhất xuống CSDL.

4.2.3. Biểu thức Lambda (`=>`) – "Trái tim" của LINQ Method Syntax

Khi viết LINQ theo cú pháp phương thức (Method Syntax), bạn sẽ thấy ký hiệu `=>` xuất hiện ở khắp mọi nơi. Ký hiệu này được gọi là **Toán tử Lambda** (đọc là "goes to" trong tiếng Anh, hoặc "suy ra", "trở thành" trong tiếng Việt).

1. Biểu thức Lambda là gì?

Về bản chất, biểu thức Lambda là một **hàm vô danh (anonymous function)** – tức là một hàm không cần đặt tên. Thay vì phải viết một phương thức dài dòng có đủ kiểu trả về, tên hàm, và cặp ngoặc nhọn `{}`, Lambda cho phép bạn viết logic của hàm đó ngay trên một dòng code cực kỳ ngắn gọn.

2. Cú pháp cơ bản:

Cấu trúc của một biểu thức Lambda bao gồm hai phần, ngăn cách nhau bởi toán tử `=>`: `(tham_số_đầu_vào) => biểu_thức_xử_ly`

3. Phân tích ví dụ thực tế:

Giả sử chúng ta có câu lệnh LINQ sau: `var cheapProducts = products.Where(p => p.Price < 50);`

Hãy "mô xê" biểu thức `p => p.Price < 50`:

- **p (Tham số đầu vào):** Đại diện cho một phần tử đơn lẻ trong tập hợp `products`.

Vì products là danh sách Product, trình biên dịch C# đủ thông minh để tự hiểu p có kiểu dữ liệu là Product. Bạn có thể đặt tên biến này là bất kỳ chữ gì (x, item, prod...), nhưng thường dùng ký tự đầu tiên của kiểu dữ liệu cho gọn.

- => (Toán tử Lambda): Đọc là "suy ra".
- p.Price < 50 (Biểu thức logic): Trả về true hoặc false.

Cách vòng lặp hoạt động ngầm: Phương thức Where sẽ tự động lặp qua từng sản phẩm trong danh sách. Tại mỗi bước lặp, nó lấy sản phẩm hiện tại gán vào biến p, rồi ném p qua bên phải mũi tên => để kiểm tra. Nếu giá < 50 (true), sản phẩm đó được giữ lại; nếu không (false), nó bị loại bỏ.

4.2.4. Các Toán Tử LINQ Thông Dụng Nhất (LINQ Operators)

Dưới đây là bảng tổng hợp các toán tử "cứu cánh" mà bạn sẽ sử dụng trong 90% các trường hợp làm việc với dữ liệu.

Toán tử (Operator)	Phân loại	Chức năng chính	Cú pháp SQL tương đương
Where	Lọc (Filtering)	Giữ lại các phần tử thỏa mãn điều kiện logic.	WHERE
Select	Chiếu (Projection)	Biến đổi dữ liệu gốc thành một hình dáng mới (ví dụ: chỉ lấy ra cột Tên từ bảng Sản phẩm).	SELECT
OrderBy	Sắp xếp (Sorting)	Sắp xếp danh sách tăng dần theo một tiêu chí.	ORDER BY ... ASC
OrderBy Descending	Sắp xếp (Sorting)	Sắp xếp danh sách giảm dần theo một tiêu chí.	ORDER BY ... DESC
GroupBy	Gom nhóm (Grouping)	Gom các phần tử có chung một đặc điểm lại thành từng nhóm.	GROUP BY
Join	Kết nối (Joining)	Ghép nối dữ liệu từ hai tập hợp khác nhau dựa trên khóa chung.	INNER JOIN

FirstOrDefault	Lấy phần tử (Element)	Lấy phần tử đầu tiên thỏa mãn điều kiện, nếu không tìm thấy trả về giá trị mặc định (null).	TOP 1
Count, Sum	Tổng hợp (Aggregation)	Đếm số lượng phần tử, hoặc tính tổng giá trị của một thuộc tính.	COUNT(), SUM()

Ví Dụ:

Để hiểu rõ hơn, hãy xem cách kết hợp các toán tử này (Method Chaining) trên một danh sách sản phẩm mẫu.

```
// Dữ liệu mẫu
var products = new List<Product>
{
    new Product { Id = 1, Name = "Laptop", Category = "Electronics", Price = 1500 },
    new Product { Id = 2, Name = "Mouse", Category = "Electronics", Price = 25 },
    new Product { Id = 3, Name = "Desk", Category = "Furniture", Price = 300 },
    new Product { Id = 4, Name = "Chair", Category = "Furniture", Price = 150 }
};

// 1. Where & Select: Lấy TÊN của các sản phẩm điện tử giá rẻ (< 100)
var cheapElectronicsNames = products
    .Where(p => p.Category == "Electronics" && p.Price < 100)
    .Select(p => p.Name) // Chỉ lấy ra thuộc tính Name (chuỗi), bỏ lại các thuộc tính khác
    .ToList();
// Kết quả: Danh sách chứa 1 phần tử ["Mouse"]

// 2. OrderBy: Sắp xếp sản phẩm theo danh mục, sau đó sắp xếp theo giá giảm dần
var sortedProducts = products
    .OrderBy(p => p.Category)
    .ThenByDescending(p => p.Price) // Dùng ThenBy cho các tiêu chí sắp xếp tiếp theo
    .ToList();
```

```
// 3. GroupBy: Đếm số lượng sản phẩm theo từng danh mục
var categoryCounts = products
    .GroupBy(p => p.Category)
    .Select(group => new
    {
        CategoryName = group.Key,
        ProductCount = group.Count()
    })
    .ToList();

// Kết quả: [{ CategoryName = "Electronics", ProductCount = 2 }, {
CategoryName = "Furniture", ProductCount = 2 }]

// 4. FirstOrDefault: Tìm sản phẩm có Id = 3
var specificProduct = products.FirstOrDefault(p => p.Id == 3);
// Kết quả: Trả về Object [Desk], an toàn không văng lỗi nếu không tìm
thấy.
```

4.2.5. Các phương thức phân trang: Skip() và Take()

Đây là hai phương thức cực kỳ mạnh mẽ của LINQ. Hãy tưởng tượng bạn có một danh sách các sản phẩm đã được sắp xếp.

- **Skip(số lượng):** Bỏ qua (nhảy qua) một số lượng phần tử đầu tiên.
- **Take(số lượng):** Lấy ra một số lượng phần tử tiếp theo.

Ví dụ minh họa: Giả sử mỗi trang có 5 sản phẩm.

- Để lấy sản phẩm cho Trang 1:
 - `products.Skip(0).Take(5)` (Bỏ qua 0, lấy 5)
- Để lấy sản phẩm cho Trang 2:
 - `products.Skip(5).Take(5)` (Bỏ qua 5 sản phẩm của trang 1, lấy 5 sản phẩm tiếp theo)
- Để lấy sản phẩm cho Trang 3:
 - `products.Skip(10).Take(5)` (Bỏ qua 10 sản phẩm của 2 trang đầu, lấy 5 sản phẩm tiếp theo)

Công thức tổng quát: `products.Skip( (pageNumber - 1) * pageSize ).Take(pageSize)`

– **Áp dụng vào SportsStore:** Trong `HomeController`, chúng ta sẽ nhận một tham

số pageNumber từ URL. Dựa vào tham số này và một biến pageSize (số sản phẩm mỗi trang) được định nghĩa sẵn, chúng ta sẽ áp dụng công thức trên vào câu lệnh LINQ để chỉ lấy về đúng số lượng sản phẩm cần thiết cho trang đó.

4.3. Custom Tag Helpers - Tạo ra các thẻ HTML của riêng bạn

4.3.1. Tại sao cần Tag Helper tùy chỉnh?

Hãy xem xét việc tạo ra các nút link phân trang. Logic của nó khá phức tạp:

1. Phải biết tổng số sản phẩm và số sản phẩm mỗi trang để tính ra tổng số trang.
2. Phải lặp để tạo ra một thẻ <a> cho mỗi trang.
3. Phải tạo đúng URL cho mỗi thẻ <a> (ví dụ: /Products?page=3).
4. Phải thêm một class CSS đặc biệt cho nút của trang hiện tại để làm nổi bật nó.

Nếu viết tất cả logic này bằng vòng lặp @for và câu lệnh @if trong file Razor, code của bạn sẽ trở nên rất **rối rắm, khó đọc và khó tái sử dụng**.

Custom Tag Helper cho phép bạn đóng gói toàn bộ logic phức tạp đó vào một class C# và "triệu hồi" nó trong View bằng một thẻ HTML đơn giản, dễ đọc.

4.3.2. Cấu trúc của một Tag Helper

1. Lớp C#:

- Phải kế thừa từ lớp TagHelper.
- Sử dụng attribute [HtmlTargetElement(...)] để chỉ định Tag Helper này sẽ được áp dụng cho thẻ HTML nào (ví dụ: một thẻ <div> có thuộc tính page-model).
- Các thuộc tính public của lớp này sẽ trở thành các **attribute** của thẻ HTML tùy chỉnh.
- Phương thức Process(context, output): Đây là "trái tim" của Tag Helper. Toàn bộ logic sinh HTML sẽ được viết ở đây.

2. Đăng ký:

- Để ứng dụng "nhận biết" được Tag Helper tùy chỉnh của bạn, bạn phải đăng ký nó trong file _ViewImports.cshtml bằng dòng lệnh @addTagHelper *, {TênAssemblyCủaProject}.

- **Áp dụng vào SportsStore:** Chúng ta sẽ tạo PageLinkTagHelper.
 - Nó sẽ được áp dụng cho thẻ `<div page-model="..." page-action="...">`.
 - Nó sẽ nhận vào một đối tượng chứa thông tin phân trang (page-model).
 - Bên trong phương thức Process, nó sẽ thực hiện toàn bộ logic phức tạp đã nêu ở trên để tự động sinh ra một loạt các thẻ `<a>` hoàn chỉnh.
 - Kết quả là trong file View, chúng ta chỉ cần viết một dòng duy nhất, cực kỳ sạch sẽ và dễ hiểu.

HƯỚNG DẪN THỰC HÀNH

Các bước:

- Tạo View Component:
 1. **(Quy ước)** Trong project WebUI, tạo một thư mục mới tên là Components.
 2. Trong thư mục này, tạo class C# `NavigationMenuViewComponent.cs`.
 3. **(Quy ước)** Trong Views/Shared, tạo một cấu trúc thư mục mới: `Components/NavigationMenu`.
 4. Trong thư mục `NavigationMenu` vừa tạo, tạo một Razor View tên là `Default.cshtml`.
 5. Để gọi View Component trong `_Layout.cshtml`, sử dụng cú pháp: `@await Component.InvokeAsync("NavigationMenu")`.
- Tạo Custom Tag Helper:
 1. **(Quy ước)** Trong project WebUI, tạo một thư mục mới tên là Infrastructure (hoặc TagHelpers).
 2. Trong thư mục này, tạo class C# `PageLinkTagHelper.cs`.
 3. Mở file `Views/_ViewImports.cshtml` và thêm dòng `@addTagHelper *, SportsStore.WebUI` (thay `SportsStore.WebUI` bằng tên Assembly của project web của bạn). Đây là bước cực kỳ quan trọng và dễ quên.

DEMO TẠO VIEW COMPONENT (NAVIGATION MENU)

Bước 1: Tạo thư mục Components

Theo quy ước tốt nhất (best practice) để giữ source code gọn gàng, chúng ta nên

gom tất cả các class xử lý logic của View Component vào một nơi.

- **Thao tác:** Trong project Web (ví dụ: WebUI hoặc thư mục gốc của project ASP.NET Core), bạn click chuột phải, chọn **Add > New Folder** và đặt tên là Components.

Bước 2: Tạo class NavigationMenuViewComponent.cs

Đây là phần "não bộ" của Component, nơi chứa logic để lấy dữ liệu (ví dụ: lấy danh sách các danh mục sản phẩm từ Database).

- **Thao tác:** Click chuột phải vào thư mục Components vừa tạo, chọn **Add > Class...** và đặt tên là NavigationMenuViewComponent.cs.
- Code mẫu:

```
using Microsoft.AspNetCore.Mvc;

namespace YourProjectName.Components
{
    // Class bắt buộc phải kế thừa từ ViewComponent
    public class NavigationMenuViewComponent : ViewComponent
    {
        // Nếu cần lấy dữ liệu từ DB, bạn có thể inject
        Repository/DbContext vào constructor ở đây

        // Phương thức Invoke (hoặc InvokeAsync) là bắt buộc.
        // ASP.NET Core sẽ tự động gọi phương thức này khi Component được
        render.
        public IViewComponentResult Invoke()
        {
            // Giả lập dữ liệu lấy từ Database
            var categories = new string[] { "WaterSports", "Soccer",
            "Chess" };

            // Truyền dữ liệu sang View (Default.cshtml)
            return View(categories);
        }
    }
}
```

Bước 3: Tạo cấu trúc thư mục chứa View

ASP.NET Core sử dụng **quy ước đặt tên (naming convention)** rất khắt khe để tìm kiếm giao diện của View Component. Nếu bạn đặt sai tên thư mục, nó sẽ báo lỗi không tìm thấy View.

- Thao tác:
 1. Mở thư mục Views > Shared.
 2. Trong Shared, tạo một thư mục mới tên là Components (Bắt buộc phải đúng chữ "Components").
 3. Trong thư mục Components, tạo tiếp một thư mục mang tên của Component, bỏ đi hậu tố "ViewComponent". Ở đây ta đặt tên là NavigationMenu.
- **Đường dẫn cuối cùng phải là:** Views/Shared/Components/NavigationMenu/

Bước 4: Tạo Razor View Default.cshtml

Đây là phần "giao diện" của Component. Mặc định, ASP.NET Core sẽ tìm file có tên là Default.cshtml.

- **Thao tác:** Click chuột phải vào thư mục NavigationMenu vừa tạo ở Bước 3, chọn **Add > View...** (chọn dạng Razor View empty) và đặt tên chính xác là Default.cshtml.
- Code mẫu:

```
@* Khai báo kiểu dữ liệu được truyền từ method Invoke() ở Bước 2 *@
@model IEnumerable<string>

<div class="d-grid gap-2">
    <a class="btn btn-outline-secondary" asp-action="Index" asp-
controller="Home" asp-route-category="">
        Home
    </a>

    *@ Lặp qua dữ liệu và render các nút menu *@
    @foreach (var category in Model)
    {
        <a class="btn btn-outline-secondary"
            asp-action="Index"
            asp-controller="Home"
            asp-route-category="@category">
```

```
        @category
    </a>
}
</div>
```

Bước 5: Gọi View Component trong _Layout.cshtml

Bây giờ Component của bạn đã hoàn chỉnh, bạn có thể "nhúng" nó vào bất kỳ đâu. Thường thì menu điều hướng sẽ được đặt trong layout chung để hiển thị ở mọi trang.

- **Thao tác:** Mở file Views/Shared/_Layout.cshtml. Tìm đến vị trí cột bên trái (sidebar) hoặc thanh điều hướng mà bạn muốn menu xuất hiện, và chèn dòng code sau:
- Code mẫu:

```
<div class="row">
    <div class="col-md-3">
        @await Component.InvokeAsync("NavigationMenu")
    </div>

    <div class="col-md-9">
        @RenderBody()
    </div>
</div>
```

Tóm lại: Khi trang web chạy, ASP.NET Core thấy lệnh `@await Component.InvokeAsync("NavigationMenu")`, nó sẽ chạy vào class `NavigationMenuViewComponent`, thực thi method `Invoke()`, lấy dữ liệu, sau đó nhét dữ liệu đó vào `Default.cshtml` để tạo ra mã HTML cuối cùng và hiển thị lên màn hình.

DEMO TẠO CUSTOM TAG HELPER (PAGE LINK)

Bước 1 (Quy ước): Tạo thư mục Infrastructure hoặc TagHelpers

Việc nhóm các file có cùng chức năng vào một thư mục giúp dự án của bạn dễ bảo trì hơn khi phình to.

- **Tại sao lại là Infrastructure?** Trong nhiều dự án, người ta dùng thư mục này để chứa các đoạn code cung cấp "cơ sở hạ tầng" hỗ trợ cho ứng dụng (như Tag Helpers, Custom Validators, Extensions...). Bạn hoàn toàn có thể đặt tên là TagHelpers cho tường minh.

- **Thao tác:** Click chuột phải vào project Web (ví dụ: WebUI), chọn **Add > New Folder** và đặt tên là Infrastructure.

Bước 2 (Quy ước): Tạo class PageLinkTagHelper.cs

Đây là nơi bạn viết code C# để định nghĩa xem thẻ HTML mới của bạn sẽ hoạt động như thế nào.

- **Quy ước đặt tên quan trọng:** ASP.NET Core tự động chuyển đổi tên class (bỏ đi chữ TagHelper ở cuối) từ dạng PascalCase sang dạng kebab-case (chữ thường, cách nhau bởi dấu gạch ngang). Do đó, class PageLinkTagHelper sẽ tương ứng với thẻ <page-link> trong HTML.
- **Thao tác:** Click chuột phải vào thư mục Infrastructure, chọn **Add > Class...** và đặt tên là PageLinkTagHelper.cs.
- Code mẫu cơ bản:

```
using Microsoft.AspNetCore.Razor.TagHelpers;

namespace YourProjectName.Infrastructure
{
    // 1. Class bắt buộc phải kế thừa từ TagHelper
    public class PageLinkTagHelper : TagHelper
    {
        // 2. Định nghĩa các thuộc tính (attribute) cho thẻ HTML.
        // Ví dụ: <page-link page-model="..." page-action="..."></page-link>
        // Bạn sẽ khai báo các property tương ứng ở đây.

        // 3. Ghi đè phương thức Process (hoặc ProcessAsync)
        // Đây là nơi bạn thao tác với thẻ HTML (thêm class, thêm thẻ con,
        // đổi nội dung...)
        public override void Process(TagHelperContext context,
        TagHelperOutput output)
        {
            // Thay thế thẻ <page-link> mặc định bằng thẻ <div> (hoặc <ul>,
            <a> tùy logic của bạn)
            output.TagName = "div";

            // Ví dụ: Tạo ra cấu trúc HTML phân trang và nhét vào
            output.Content
```

```
        // ... (Logic tạo các thẻ <a> phân trang sẽ nằm ở đây)
    }
}
}
```

Bước 3 (Quy ước): Đăng ký Tag Helper trong _ViewImports.cshtml

Lưu ý: Đây là bước mà 99% người mới học thường quên. Nếu bạn không làm bước này, khi bạn gõ <page-link> vào file HTML, trình duyệt sẽ coi nó như một thẻ HTML vô nghĩa và bỏ qua nó, không có lỗi nào được báo ra cả.

- **Mục đích:** File _ViewImports.cshtml chạy trước tất cả các file View khác. Dòng lệnh bạn sắp thêm vào sẽ nói với ASP.NET Core rằng: "Hãy quét toàn bộ Assembly (project) này, tìm tất cả những class nào là Tag Helper và kích hoạt chúng lên để tôi dùng trong HTML".
- Thao tác:
 1. Mở thư mục Views.
 2. Mở file _ViewImports.cshtml.
 3. Thêm dòng lệnh sau vào (dưới các lệnh @using hiện có):

```
@* Dấu * nghĩa là áp dụng cho tất cả Tag Helper *@
@* Phía sau dấu phẩy là Tên Assembly (thường là tên project của bạn) *@
@addTagHelper *, YourProjectName
```

*(Lưu ý: Nếu project của bạn tên là SportsStore.WebUI, thì dòng đó sẽ là @addTagHelper *, SportsStore.WebUI).*

Tóm lại luồng hoạt động: Khi hoàn tất 3 bước này, bất cứ khi nào bạn viết <page-link> trong file Razor (ví dụ Index.cshtml), ASP.NET Core sẽ kiểm tra _ViewImports, phát hiện ra PageLinkTagHelper đã được đăng ký, sau đó nó sẽ chạy method Process() trong file C# của bạn để render ra đoạn mã HTML phân trang hoàn chỉnh.

CÂU HỎI LÝ THUYẾT

1. Tình huống nào sau đây nên sử dụng View Component thay vì Partial View: hiển thị địa chỉ công ty ở footer hay hiển thị 5 bài viết mới nhất từ blog? Tại sao?
2. Một danh sách có 50 sản phẩm. pageSize là 8. Để lấy các sản phẩm cho trang thứ 4, phương thức Skip() và Take() cần nhận tham số là bao nhiêu?

3. Mục đích chính của việc tạo ra Custom Tag Helper là gì?
4. Để một Custom Tag Helper hoạt động, ngoài việc tạo class C#, chúng ta cần phải làm gì trong project?

TỔNG KẾT BÀI HỌC

Tuần này, chúng ta đã trang bị cho ứng dụng những công cụ mạnh mẽ để xử lý các kịch bản thực tế:

- **View Components** giúp chúng ta xây dựng các thành phần UI độc lập, có thể tái sử dụng, làm cho code trở nên module hóa và dễ quản lý hơn.
- **Phân trang với LINQ** là một kỹ thuật thiết yếu để đảm bảo ứng dụng hoạt động hiệu quả với lượng dữ liệu lớn.
- **Custom Tag Helpers** là một kỹ thuật nâng cao giúp chúng ta giữ cho mã nguồn trong View luôn sạch sẽ, dễ đọc bằng cách đóng gói các logic phức tạp.

Giờ đây, ứng dụng của chúng ta không chỉ "đẹp" mà còn "thông minh" và "hiệu quả".

BÀI TẬP VỀ NHÀ

1. Làm nổi bật danh mục đang chọn:
 - Trong `NavigationMenuViewComponent`, hãy tìm cách lấy ra danh mục hiện tại đang được chọn từ URL. (Gợi ý: sử dụng `ViewBag` hoặc thuộc tính `RouteData` có sẵn trong View Component).
 - Truyền thông tin này sang `View Default.cshtml`.
 - Trong `View Default.cshtml`, sử dụng một câu lệnh `@if` để so sánh danh mục của nút với danh mục đang được chọn. Nếu trùng khớp, hãy thêm một class CSS khác (ví dụ `btn-primary`) để làm nổi bật nó.
2. Tùy chỉnh Tag Helper Phân trang:
 - Thêm hai thuộc tính public mới vào `PageLinkTagHelper.cs`: `public string PageClass { get; set; }` và `public string PageClassSelected { get; set; }`.
 - Trong phương thức `Process`, sử dụng giá trị từ các thuộc tính này thay vì hard-code các class CSS `btn-primary` và `btn-outline-primary`.
 - Trong `Index.cshtml`, thử truyền các giá trị class Bootstrap khác vào Tag Helper

để xem kết quả, ví dụ: `<div ... page-class="btn-outline-info" page-class-selected="btn-info"></div>`.

TÀI LIỆU THAM KHẢO

- [1]. View Components in ASP.NET Core: Microsoft Docs - View Components
- [2]. LINQ Query Examples (Skip, Take): LINQ Samples - Skip & Take
- [3]. Author Tag Helpers in ASP.NET Core: Microsoft Docs - Authoring Tag Helpers